

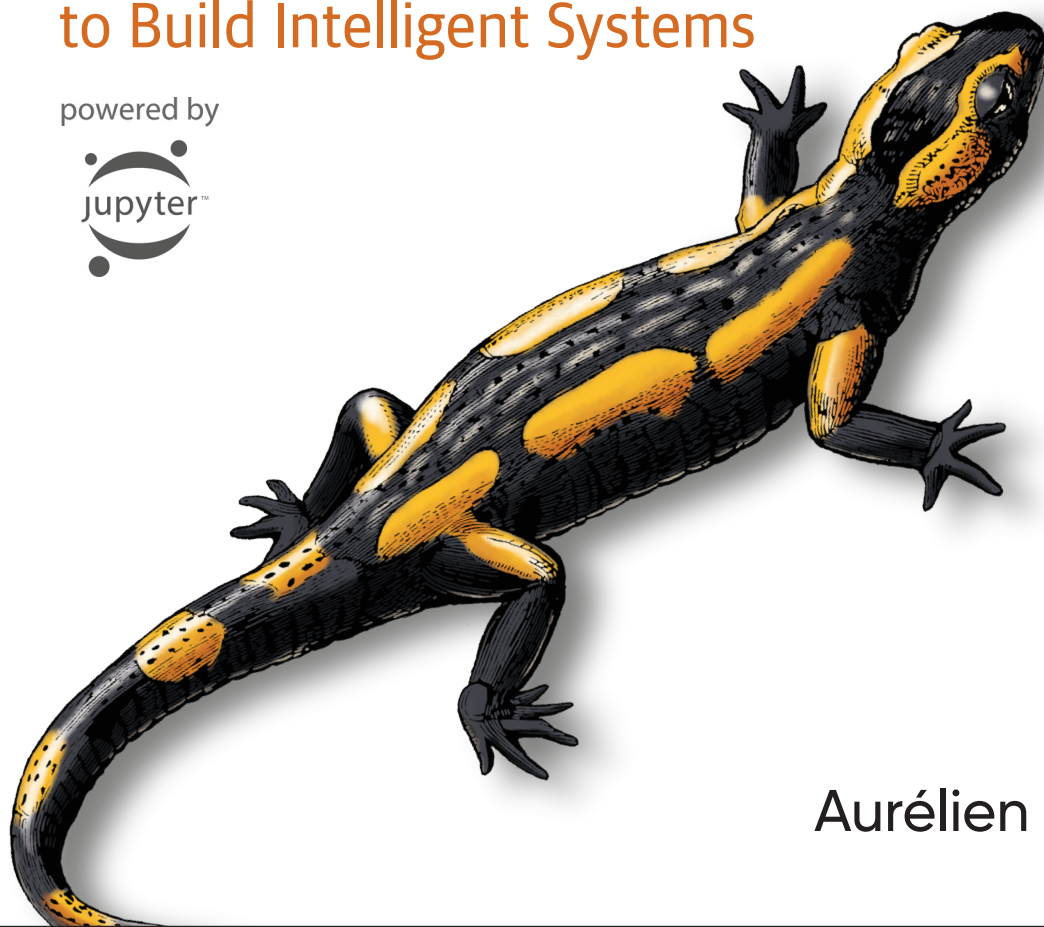
O'REILLY®

2nd Edition
Updated for
TensorFlow 2

Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow

Concepts, Tools, and Techniques
to Build Intelligent Systems

powered by



Aurélien Géron

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow

Through a series of breakthroughs, Deep Learning has boosted the entire field of machine learning. Now, even programmers who know close to nothing about this technology can use simple, efficient tools to implement programs capable of learning from data. The updated edition of this best-selling book uses concrete examples, minimal theory, and production-ready Python frameworks to help you gain an intuitive understanding of the concepts and tools for building intelligent systems.

You'll learn a range of techniques that you can quickly put to use. With exercises in each chapter to help you apply what you've learned, all you need is programming experience to get started. All code is available on GitHub. It has been updated to TensorFlow 2 and the latest version of Scikit-Learn.

- Learn Machine Learning fundamentals through an end-to-end project using Scikit-Learn and pandas
- Build and train many neural network architectures for classification and regression using TensorFlow 2
- Discover object detection, semantic segmentation, attention mechanisms, language models, GANs, and more
- Explore the Keras API, the official high-level API for TensorFlow 2
- Productionize TensorFlow models using TensorFlow's Data API, distribution strategies API, TF Transform, and TF-Serving
- Deploy on Google Cloud AI Platform or on mobile devices
- Exploit unsupervised learning techniques such as dimensionality reduction, clustering, and anomaly detection
- Create autonomous learning agents with Reinforcement Learning, including using the TF-Agents library

"An exceptional resource to study Machine Learning. You will find clear-minded, intuitive explanations, and a wealth of practical tips."

—François Chollet

Author of Keras, author of
Deep Learning with Python

"This book is a great introduction to the theory and practice of solving problems with neural networks; I recommend it to anyone interested in learning about practical ML."

—Pete Warden

Mobile Lead for TensorFlow

Aurélien Géron is a machine learning consultant and trainer. A former Googler, he led YouTube's video classification team from 2013 to 2016. He was also a founder and CTO of Wifirst (a leading Wireless ISP in France) from 2002 to 2012.

DATA | DATA SCIENCE | DATA ANALYTICS
MACHINE LEARNING | DEEP LEARNING
PYTHON MACHINE LEARNING

US \$74.99

CAN \$98.99

ISBN: 978-1-492-03264-9



Twitter: @oreillymedia
facebook.com/oreilly

SECOND EDITION

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow

*Concepts, Tools, and Techniques to
Build Intelligent Systems*

Aurélien Géron

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow

by Aurélien Geron

Copyright © 2019 Kiwisoft S.A.S. All rights reserved.

Printed in Canada.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Editors: Rachel Roumeliotis and Nicole Tache

Production Editor: Kristen Brown

Copyeditor: Amanda Kersey

Proofreader: Rachel Head

Indexer: Judith McConville

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Rebecca Demarest

September 2019: Second Edition

Revision History for the Second Edition

2019-09-05: First Release

2019-10-11: Second Release

2019-11-22: Third Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781492032649> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author, and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-492-03264-9

[TI]

Table of Contents

Preface.....	xv
--------------	----

Part I. The Fundamentals of Machine Learning

1. The Machine Learning Landscape.....	1
What Is Machine Learning?	2
Why Use Machine Learning?	2
Examples of Applications	5
Types of Machine Learning Systems	7
Supervised/Unsupervised Learning	7
Batch and Online Learning	14
Instance-Based Versus Model-Based Learning	17
Main Challenges of Machine Learning	23
Insufficient Quantity of Training Data	23
Nonrepresentative Training Data	25
Poor-Quality Data	26
Irrelevant Features	27
Overfitting the Training Data	27
Underfitting the Training Data	29
Stepping Back	30
Testing and Validating	30
Hyperparameter Tuning and Model Selection	31
Data Mismatch	32
Exercises	33
2. End-to-End Machine Learning Project.....	35
Working with Real Data	35

Look at the Big Picture	37
Frame the Problem	37
Select a Performance Measure	39
Check the Assumptions	42
Get the Data	42
Create the Workspace	42
Download the Data	46
Take a Quick Look at the Data Structure	47
Create a Test Set	51
Discover and Visualize the Data to Gain Insights	56
Visualizing Geographical Data	56
Looking for Correlations	58
Experimenting with Attribute Combinations	61
Prepare the Data for Machine Learning Algorithms	62
Data Cleaning	63
Handling Text and Categorical Attributes	65
Custom Transformers	68
Feature Scaling	69
Transformation Pipelines	70
Select and Train a Model	72
Training and Evaluating on the Training Set	72
Better Evaluation Using Cross-Validation	73
Fine-Tune Your Model	75
Grid Search	76
Randomized Search	78
Ensemble Methods	78
Analyze the Best Models and Their Errors	78
Evaluate Your System on the Test Set	79
Launch, Monitor, and Maintain Your System	80
Try It Out!	83
Exercises	84
3. Classification.....	85
MNIST	85
Training a Binary Classifier	88
Performance Measures	88
Measuring Accuracy Using Cross-Validation	89
Confusion Matrix	90
Precision and Recall	92
Precision/Recall Trade-off	93
The ROC Curve	97
Multiclass Classification	100

Error Analysis	102
Multilabel Classification	106
Multioutput Classification	107
Exercises	108
4. Training Models.....	111
Linear Regression	112
The Normal Equation	114
Computational Complexity	117
Gradient Descent	118
Batch Gradient Descent	121
Stochastic Gradient Descent	124
Mini-batch Gradient Descent	127
Polynomial Regression	128
Learning Curves	130
Regularized Linear Models	134
Ridge Regression	135
Lasso Regression	137
Elastic Net	140
Early Stopping	141
Logistic Regression	142
Estimating Probabilities	143
Training and Cost Function	144
Decision Boundaries	145
Softmax Regression	148
Exercises	151
5. Support Vector Machines.....	153
Linear SVM Classification	153
Soft Margin Classification	154
Nonlinear SVM Classification	157
Polynomial Kernel	158
Similarity Features	159
Gaussian RBF Kernel	160
Computational Complexity	162
SVM Regression	162
Under the Hood	164
Decision Function and Predictions	165
Training Objective	166
Quadratic Programming	167
The Dual Problem	168
Kernelized SVMs	169

Online SVMs	172
Exercises	174
6. Decision Trees.....	175
Training and Visualizing a Decision Tree	175
Making Predictions	176
Estimating Class Probabilities	178
The CART Training Algorithm	179
Computational Complexity	180
Gini Impurity or Entropy?	180
Regularization Hyperparameters	181
Regression	183
Instability	185
Exercises	186
7. Ensemble Learning and Random Forests.....	189
Voting Classifiers	189
Bagging and Pasting	192
Bagging and Pasting in Scikit-Learn	194
Out-of-Bag Evaluation	195
Random Patches and Random Subspaces	196
Random Forests	197
Extra-Trees	198
Feature Importance	198
Boosting	199
AdaBoost	200
Gradient Boosting	203
Stacking	208
Exercises	211
8. Dimensionality Reduction.....	213
The Curse of Dimensionality	214
Main Approaches for Dimensionality Reduction	215
Projection	215
Manifold Learning	218
PCA	219
Preserving the Variance	219
Principal Components	220
Projecting Down to d Dimensions	221
Using Scikit-Learn	222
Explained Variance Ratio	222
Choosing the Right Number of Dimensions	223

PCA for Compression	224
Randomized PCA	225
Incremental PCA	225
Kernel PCA	226
Selecting a Kernel and Tuning Hyperparameters	227
LLE	230
Other Dimensionality Reduction Techniques	232
Exercises	233
9. Unsupervised Learning Techniques.....	235
Clustering	236
K-Means	238
Limits of K-Means	248
Using Clustering for Image Segmentation	249
Using Clustering for Preprocessing	251
Using Clustering for Semi-Supervised Learning	253
DBSCAN	255
Other Clustering Algorithms	258
Gaussian Mixtures	260
Anomaly Detection Using Gaussian Mixtures	266
Selecting the Number of Clusters	267
Bayesian Gaussian Mixture Models	270
Other Algorithms for Anomaly and Novelty Detection	274
Exercises	275

Part II. Neural Networks and Deep Learning

10. Introduction to Artificial Neural Networks with Keras.....	279
From Biological to Artificial Neurons	280
Biological Neurons	281
Logical Computations with Neurons	283
The Perceptron	284
The Multilayer Perceptron and Backpropagation	289
Regression MLPs	292
Classification MLPs	294
Implementing MLPs with Keras	295
Installing TensorFlow 2	296
Building an Image Classifier Using the Sequential API	297
Building a Regression MLP Using the Sequential API	307
Building Complex Models Using the Functional API	308
Using the Subclassing API to Build Dynamic Models	313

Saving and Restoring a Model	314
Using Callbacks	315
Using TensorBoard for Visualization	317
Fine-Tuning Neural Network Hyperparameters	320
Number of Hidden Layers	323
Number of Neurons per Hidden Layer	324
Learning Rate, Batch Size, and Other Hyperparameters	325
Exercises	327
11. Training Deep Neural Networks.....	331
The Vanishing/Exploding Gradients Problems	332
Glorot and He Initialization	333
Nonsaturating Activation Functions	335
Batch Normalization	338
Gradient Clipping	345
Reusing Pretrained Layers	345
Transfer Learning with Keras	347
Unsupervised Pretraining	349
Pretraining on an Auxiliary Task	350
Faster Optimizers	351
Momentum Optimization	351
Nesterov Accelerated Gradient	353
AdaGrad	354
RMSProp	355
Adam and Nadam Optimization	356
Learning Rate Scheduling	359
Avoiding Overfitting Through Regularization	364
ℓ_1 and ℓ_2 Regularization	364
Dropout	365
Monte Carlo (MC) Dropout	368
Max-Norm Regularization	370
Summary and Practical Guidelines	371
Exercises	373
12. Custom Models and Training with TensorFlow.....	375
A Quick Tour of TensorFlow	376
Using TensorFlow like NumPy	379
Tensors and Operations	379
Tensors and NumPy	381
Type Conversions	381
Variables	382
Other Data Structures	383

Customizing Models and Training Algorithms	384
Custom Loss Functions	384
Saving and Loading Models That Contain Custom Components	385
Custom Activation Functions, Initializers, Regularizers, and Constraints	387
Custom Metrics	388
Custom Layers	391
Custom Models	394
Losses and Metrics Based on Model Internals	397
Computing Gradients Using Autodiff	399
Custom Training Loops	402
TensorFlow Functions and Graphs	405
AutoGraph and Tracing	407
TF Function Rules	409
Exercises	410
13. Loading and Preprocessing Data with TensorFlow.....	413
The Data API	414
Chaining Transformations	415
Shuffling the Data	416
Preprocessing the Data	419
Putting Everything Together	420
Prefetching	421
Using the Dataset with tf.keras	423
The TFRecord Format	424
Compressed TFRecord Files	425
A Brief Introduction to Protocol Buffers	425
TensorFlow Protobufs	427
Loading and Parsing Examples	428
Handling Lists of Lists Using the SequenceExample Protobuf	429
Preprocessing the Input Features	430
Encoding Categorical Features Using One-Hot Vectors	431
Encoding Categorical Features Using Embeddings	433
Keras Preprocessing Layers	437
TF Transform	439
The TensorFlow Datasets (TFDS) Project	441
Exercises	442
14. Deep Computer Vision Using Convolutional Neural Networks.....	445
The Architecture of the Visual Cortex	446
Convolutional Layers	448
Filters	450
Stacking Multiple Feature Maps	451

TensorFlow Implementation	453
Memory Requirements	456
Pooling Layers	456
TensorFlow Implementation	458
CNN Architectures	460
LeNet-5	463
AlexNet	464
GoogLeNet	466
VGGNet	470
ResNet	471
Xception	474
SENet	476
Implementing a ResNet-34 CNN Using Keras	478
Using Pretrained Models from Keras	479
Pretrained Models for Transfer Learning	481
Classification and Localization	483
Object Detection	485
Fully Convolutional Networks	487
You Only Look Once (YOLO)	489
Semantic Segmentation	492
Exercises	496
15. Processing Sequences Using RNNs and CNNs.	497
Recurrent Neurons and Layers	498
Memory Cells	500
Input and Output Sequences	501
Training RNNs	502
Forecasting a Time Series	503
Baseline Metrics	505
Implementing a Simple RNN	505
Deep RNNs	506
Forecasting Several Time Steps Ahead	508
Handling Long Sequences	511
Fighting the Unstable Gradients Problem	512
Tackling the Short-Term Memory Problem	514
Exercises	523
16. Natural Language Processing with RNNs and Attention.	525
Generating Shakespearean Text Using a Character RNN	526
Creating the Training Dataset	527
How to Split a Sequential Dataset	527
Chopping the Sequential Dataset into Multiple Windows	528

Building and Training the Char-RNN Model	530
Using the Char-RNN Model	531
Generating Fake Shakespearean Text	531
Stateful RNN	532
Sentiment Analysis	534
Masking	538
Reusing Pretrained Embeddings	540
An Encoder–Decoder Network for Neural Machine Translation	542
Bidirectional RNNs	546
Beam Search	547
Attention Mechanisms	549
Visual Attention	552
Attention Is All You Need: The Transformer Architecture	554
Recent Innovations in Language Models	563
Exercises	565
17. Representation Learning and Generative Learning Using Autoencoders and GANs.	567
Efficient Data Representations	569
Performing PCA with an Undercomplete Linear Autoencoder	570
Stacked Autoencoders	572
Implementing a Stacked Autoencoder Using Keras	572
Visualizing the Reconstructions	574
Visualizing the Fashion MNIST Dataset	574
Unsupervised Pretraining Using Stacked Autoencoders	576
Tying Weights	577
Training One Autoencoder at a Time	578
Convolutional Autoencoders	579
Recurrent Autoencoders	580
Denoising Autoencoders	581
Sparse Autoencoders	582
Variational Autoencoders	586
Generating Fashion MNIST Images	590
Generative Adversarial Networks	592
The Difficulties of Training GANs	596
Deep Convolutional GANs	598
Progressive Growing of GANs	601
StyleGANs	604
Exercises	607
18. Reinforcement Learning.	609
Learning to Optimize Rewards	610
Policy Search	612

Introduction to OpenAI Gym	613
Neural Network Policies	617
Evaluating Actions: The Credit Assignment Problem	619
Policy Gradients	620
Markov Decision Processes	625
Temporal Difference Learning	629
Q-Learning	630
Exploration Policies	632
Approximate Q-Learning and Deep Q-Learning	633
Implementing Deep Q-Learning	634
Deep Q-Learning Variants	639
Fixed Q-Value Targets	639
Double DQN	640
Prioritized Experience Replay	640
Dueling DQN	641
The TF-Agents Library	642
Installing TF-Agents	643
TF-Agents Environments	643
Environment Specifications	644
Environment Wrappers and Atari Preprocessing	645
Training Architecture	649
Creating the Deep Q-Network	650
Creating the DQN Agent	652
Creating the Replay Buffer and the Corresponding Observer	654
Creating Training Metrics	655
Creating the Collect Driver	656
Creating the Dataset	658
Creating the Training Loop	661
Overview of Some Popular RL Algorithms	662
Exercises	664
19. Training and Deploying TensorFlow Models at Scale.	667
Serving a TensorFlow Model	668
Using TensorFlow Serving	668
Creating a Prediction Service on GCP AI Platform	677
Using the Prediction Service	682
Deploying a Model to a Mobile or Embedded Device	685
Using GPUs to Speed Up Computations	689
Getting Your Own GPU	690
Using a GPU-Equipped Virtual Machine	692
Colaboratory	693
Managing the GPU RAM	694

Placing Operations and Variables on Devices	697
Parallel Execution Across Multiple Devices	699
Training Models Across Multiple Devices	701
Model Parallelism	701
Data Parallelism	704
Training at Scale Using the Distribution Strategies API	709
Training a Model on a TensorFlow Cluster	711
Running Large Training Jobs on Google Cloud AI Platform	714
Black Box Hyperparameter Tuning on AI Platform	716
Exercises	717
Thank You!	718
A. Exercise Solutions.....	719
B. Machine Learning Project Checklist.....	755
C. SVM Dual Problem.....	761
D. Autodiff.....	765
E. Other Popular ANN Architectures.....	773
F. Special Data Structures.....	783
G. TensorFlow Graphs.....	791
Index.....	801

The Machine Learning Tsunami

In 2006, Geoffrey Hinton et al. published [a paper](#)¹ showing how to train a deep neural network capable of recognizing handwritten digits with state-of-the-art precision (>98%). They branded this technique “Deep Learning.” A deep neural network is a (very) simplified model of our cerebral cortex, composed of a stack of layers of artificial neurons. Training a deep neural net was widely considered impossible at the time,² and most researchers had abandoned the idea in the late 1990s. This paper revived the interest of the scientific community, and before long many new papers demonstrated that Deep Learning was not only possible, but capable of mind-blowing achievements that no other Machine Learning (ML) technique could hope to match (with the help of tremendous computing power and great amounts of data). This enthusiasm soon extended to many other areas of Machine Learning.

A decade or so later, Machine Learning has conquered the industry: it is at the heart of much of the magic in today’s high-tech products, ranking your web search results, powering your smartphone’s speech recognition, recommending videos, and beating the world champion at the game of Go. Before you know it, it will be driving your car.

Machine Learning in Your Projects

So, naturally you are excited about Machine Learning and would love to join the party!

1 Geoffrey E. Hinton et al., “A Fast Learning Algorithm for Deep Belief Nets,” *Neural Computation* 18 (2006): 1527–1554.

2 Despite the fact that Yann LeCun’s deep convolutional neural networks had worked well for image recognition since the 1990s, although they were not as general-purpose.

Perhaps you would like to give your homemade robot a brain of its own? Make it recognize faces? Or learn to walk around?

Or maybe your company has tons of data (user logs, financial data, production data, machine sensor data, hotline stats, HR reports, etc.), and more than likely you could unearth some hidden gems if you just knew where to look. With Machine Learning, you could accomplish the following **and more**:

- Segment customers and find the best marketing strategy for each group.
- Recommend products for each client based on what similar clients bought.
- Detect which transactions are likely to be fraudulent.
- Forecast next year's revenue.

Whatever the reason, you have decided to learn Machine Learning and implement it in your projects. Great idea!

Objective and Approach

This book assumes that you know close to nothing about Machine Learning. Its goal is to give you the concepts, tools, and intuition you need to implement programs capable of *learning from data*.

We will cover a large number of techniques, from the simplest and most commonly used (such as Linear Regression) to some of the Deep Learning techniques that regularly win competitions.

Rather than implementing our own toy versions of each algorithm, we will be using production-ready Python frameworks:

- **Scikit-Learn** is very easy to use, yet it implements many Machine Learning algorithms efficiently, so it makes for a great entry point to learning Machine Learning. It was created by David Cournapeau in 2007, and is now led by a team of researchers at the French Institute for Research in Computer Science and Automation (Inria).
- **TensorFlow** is a more complex library for distributed numerical computation. It makes it possible to train and run very large neural networks efficiently by distributing the computations across potentially hundreds of multi-GPU (graphics processing unit) servers. TensorFlow (TF) was created at Google and supports many of its large-scale Machine Learning applications. It was open sourced in November 2015, and version 2.0 was released in September 2019.
- **Keras** is a high-level Deep Learning API that makes it very simple to train and run neural networks. It can run on top of either TensorFlow, Theano, or Microsoft Cognitive Toolkit (formerly known as CNTK). TensorFlow comes with its

own implementation of this API, called *tf.keras*, which provides support for some advanced TensorFlow features (e.g., the ability to efficiently load data).

The book favors a hands-on approach, growing an intuitive understanding of Machine Learning through concrete working examples and just a little bit of theory. While you can read this book without picking up your laptop, I highly recommend you experiment with the code examples available online as Jupyter notebooks at <https://github.com/ageron/handson-ml2>.

Prerequisites

This book assumes that you have some Python programming experience and that you are familiar with Python's main scientific libraries—in particular, **NumPy**, **pandas**, and **Matplotlib**.

Also, if you care about what's under the hood, you should have a reasonable understanding of college-level math as well (calculus, linear algebra, probabilities, and statistics).

If you don't know Python yet, <http://learnpython.org/> is a great place to start. The official tutorial on **Python.org** is also quite good.

If you have never used Jupyter, **Chapter 2** will guide you through installation and the basics: it is a powerful tool to have in your toolbox.

If you are not familiar with Python's scientific libraries, the provided Jupyter notebooks include a few tutorials. There is also a quick math tutorial for linear algebra.

Roadmap

This book is organized in two parts. **Part I, *The Fundamentals of Machine Learning***, covers the following topics:

- What Machine Learning is, what problems it tries to solve, and the main categories and fundamental concepts of its systems
- The steps in a typical Machine Learning project
- Learning by fitting a model to data
- Optimizing a cost function
- Handling, cleaning, and preparing data
- Selecting and engineering features
- Selecting a model and tuning hyperparameters using cross-validation
- The challenges of Machine Learning, in particular underfitting and overfitting (the bias/variance trade-off)

- The most common learning algorithms: Linear and Polynomial Regression, Logistic Regression, k-Nearest Neighbors, Support Vector Machines, Decision Trees, Random Forests, and Ensemble methods
- Reducing the dimensionality of the training data to fight the “curse of dimensionality”
- Other unsupervised learning techniques, including clustering, density estimation, and anomaly detection

Part II, Neural Networks and Deep Learning, covers the following topics:

- What neural nets are and what they’re good for
- Building and training neural nets using TensorFlow and Keras
- The most important neural net architectures: feedforward neural nets for tabular data, convolutional nets for computer vision, recurrent nets and long short-term memory (LSTM) nets for sequence processing, encoder/decoders and Transformers for natural language processing, autoencoders and generative adversarial networks (GANs) for generative learning
- Techniques for training deep neural nets
- How to build an agent (e.g., a bot in a game) that can learn good strategies through trial and error, using Reinforcement Learning
- Loading and preprocessing large amounts of data efficiently
- Training and deploying TensorFlow models at scale

The first part is based mostly on Scikit-Learn, while the second part uses TensorFlow and Keras.



Don’t jump into deep waters too hastily: while Deep Learning is no doubt one of the most exciting areas in Machine Learning, you should master the fundamentals first. Moreover, most problems can be solved quite well using simpler techniques such as Random Forests and Ensemble methods (discussed in [Part I](#)). Deep Learning is best suited for complex problems such as image recognition, speech recognition, or natural language processing, provided you have enough data, computing power, and patience.

Changes in the Second Edition

This second edition has six main objectives:

1. Cover additional ML topics: more unsupervised learning techniques (including clustering, anomaly detection, density estimation, and mixture models); more techniques for training deep nets (including self-normalized networks); additional computer vision techniques (including Xception, SNet, object detection with YOLO, and semantic segmentation using R-CNN); handling sequences using convolutional neural networks (CNNs, including WaveNet); natural language processing using recurrent neural networks (RNNs), CNNs, and Transformers; and GANs.
2. Cover additional libraries and APIs (Keras, the Data API, TF-Agents for Reinforcement Learning) and training and deploying TF models at scale using the Distribution Strategies API, TF-Serving, and Google Cloud AI Platform. Also briefly introduce TF Transform, TFLite, TF Addons/Seq2Seq, and TensorFlow.js.
3. Discuss some of the latest important results from Deep Learning research.
4. Migrate all TensorFlow chapters to TensorFlow 2, and use TensorFlow's implementation of the Keras API (tf.keras) whenever possible.
5. Update the code examples to use the latest versions of Scikit-Learn, NumPy, pandas, Matplotlib, and other libraries.
6. Clarify some sections and fix some errors, thanks to plenty of great feedback from readers.

Some chapters were added, others were rewritten, and a few were reordered. See <https://homl.info/changes2> for more details on what changed in the second edition.

Other Resources

Many excellent resources are available to learn about Machine Learning. For example, Andrew Ng's [ML course on Coursera](#) is amazing, although it requires a significant time investment (think months).

There are also many interesting websites about Machine Learning, including of course Scikit-Learn's exceptional [User Guide](#). You may also enjoy [Dataquest](#), which provides very nice interactive tutorials, and ML blogs such as those listed on [Quora](#). Finally, the [Deep Learning website](#) has a good list of resources to check out to learn more.

There are many other introductory books about Machine Learning. In particular:

- Joel Grus’s *Data Science from Scratch* (O’Reilly) presents the fundamentals of Machine Learning and implements some of the main algorithms in pure Python (from scratch, as the name suggests).
- Stephen Marsland’s *Machine Learning: An Algorithmic Perspective* (Chapman & Hall) is a great introduction to Machine Learning, covering a wide range of topics in depth with code examples in Python (also from scratch, but using NumPy).
- Sebastian Raschka’s *Python Machine Learning* (Packt Publishing) is also a great introduction to Machine Learning and leverages Python open source libraries (Pylearn 2 and Theano).
- François Chollet’s *Deep Learning with Python* (Manning) is a very practical book that covers a large range of topics in a clear and concise way, as you might expect from the author of the excellent Keras library. It favors code examples over mathematical theory.
- Andriy Burkov’s *The Hundred-Page Machine Learning Book* is very short and covers an impressive range of topics, introducing them in approachable terms without shying away from the math equations.
- Yaser S. Abu-Mostafa, Malik Magdon-Ismael, and Hsuan-Tien Lin’s *Learning from Data* (AMLBook) is a rather theoretical approach to ML that provides deep insights, in particular on the bias/variance trade-off (see [Chapter 4](#)).
- Stuart Russell and Peter Norvig’s *Artificial Intelligence: A Modern Approach*, 3rd Edition (Pearson), is a great (and huge) book covering an incredible amount of topics, including Machine Learning. It helps put ML into perspective.

Finally, joining ML competition websites such as [Kaggle.com](#) will allow you to practice your skills on real-world problems, with help and insights from some of the best ML professionals out there.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates a warning or caution.

Code Examples

There is a series of Jupyter notebooks full of supplemental material, such as code examples and exercises, available for download at <https://github.com/ageron/handson-ml2>.

Some of the code examples in the book leave out repetitive sections or details that are obvious or unrelated to Machine Learning. This keeps the focus on the important parts of the code and saves space to cover more topics. If you want the full code examples, they are all available in the Jupyter notebooks.

Note that when the code examples display some outputs, these code examples are shown with Python prompts (`>>>` and `...`), as in a Python shell, to clearly distinguish the code from the outputs. For example, this code defines the `square()` function, then it computes and displays the square of 3:

```
>>> def square(x):  
...     return x ** 2  
...  
>>> result = square(3)  
>>> result  
9
```

When code does not display anything, prompts are not used. However, the result may sometimes be shown as a comment, like this:

```
def square(x):  
    return x ** 2  
  
result = square(3) # result is 9
```

Using Code Examples

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you’re reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing a CD-ROM of examples from O’Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product’s documentation does require permission.

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd Edition, by Aurélien Géron (O’Reilly). Copyright 2019 Kiwisoft S.A.S., 978-1-492-03264-9.” If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O’Reilly Online Learning



For almost 40 years, *O’Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, conferences, and our online learning platform. O’Reilly’s online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O’Reilly and 200+ other publishers. For more information, please visit <http://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international or local)
707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://homl.info/oreilly2>.

To comment or ask technical questions about this book, send email to bookquestions@oreilly.com.

For more information about our books, courses, conferences, and news, see our website at <http://www.oreilly.com>.

Find us on Facebook: <http://facebook.com/oreilly>

Follow us on Twitter: <http://twitter.com/oreillymedia>

Watch us on YouTube: <http://www.youtube.com/oreillymedia>

Acknowledgments

Never in my wildest dreams did I imagine that the first edition of this book would get such a large audience. I received so many messages from readers, many asking questions, some kindly pointing out errata, and most sending me encouraging words. I cannot express how grateful I am to all these readers for their tremendous support. Thank you all so very much! Please do not hesitate to [file issues on GitHub](#) if you find errors in the code examples (or just to ask questions), or to submit [errata](#) if you find errors in the text. Some readers also shared how this book helped them get their first job, or how it helped them solve a concrete problem they were working on. I find such feedback incredibly motivating. If you find this book helpful, I would love it if you could share your story with me, either privately (e.g., via [LinkedIn](#)) or publicly (e.g., in a tweet or through an [Amazon review](#)).

I am also incredibly thankful to all the amazing people who took time out of their busy lives to review my book with such care. In particular, I would like to thank François Chollet for reviewing all the chapters based on Keras and TensorFlow and giving me some great in-depth feedback. Since Keras is one of the main additions to this second edition, having its author review the book was invaluable. I highly recommend

François's book *Deep Learning with Python* (Manning): it has the conciseness, clarity, and depth of the Keras library itself. Special thanks as well to Ankur Patel, who reviewed every chapter of this second edition and gave me excellent feedback, in particular on **Chapter 9**, which covers unsupervised learning techniques. He could write a whole book on the topic... oh, wait, he did! Do check out *Hands-On Unsupervised Learning Using Python: How to Build Applied Machine Learning Solutions from Unlabeled Data* (O'Reilly). Huge thanks as well to Olzhas Akpambetov, who reviewed all the chapters in the second part of the book, tested much of the code, and offered many great suggestions. I'm grateful to Mark Daoust, Jon Krohn, Dominic Monn, and Josh Patterson for reviewing the second part of this book so thoroughly and offering their expertise. They left no stone unturned and provided amazingly useful feedback.

While writing this second edition, I was fortunate enough to get plenty of help from members of the TensorFlow team—in particular Martin Wicke, who tirelessly answered dozens of my questions and dispatched the rest to the right people, including Karmel Allison, Paige Bailey, Eugene Brevdo, William Chargin, Daniel “Wolff” Dobson, Nick Felt, Bruce Fontaine, Goldie Gadde, Sandeep Gupta, Priya Gupta, Kevin Haas, Konstantinos Katsiapis, Viacheslav Kovalevskyi, Allen Lavoie, Clemens Mewald, Dan Moldovan, Sean Morgan, Tom O'Malley, Alexandre Passos, André Susano Pinto, Anthony Platanios, Oscar Ramirez, Anna Revinskaya, Saurabh Saxena, Ryan Sepassi, Jiri Simsa, Xiaodan Song, Christina Sorokin, Dustin Tran, Todd Wang, Pete Warden (who also reviewed the first edition) Edd Wilder-James, and Yuefeng Zhou, all of whom were tremendously helpful. Huge thanks to all of you, and to all other members of the TensorFlow team, not just for your help, but also for making such a great library! Special thanks to Irene Giannoumis and Robert Crowe of the TFX team for reviewing **Chapters 13** and **19** in depth.

Many thanks as well to O'Reilly's fantastic staff, in particular Nicole Taché, who gave me insightful feedback and was always cheerful, encouraging, and helpful: I could not dream of a better editor. Big thanks to Michele Cronin as well, who was very helpful (and patient) at the start of this second edition, and to Kristen Brown, the production editor for the second edition, who saw it through all the steps (she also coordinated fixes and updates for each reprint of the first edition). Thanks as well to Rachel Monaghan and Amanda Kersey for their thorough copyediting (respectively for the first and second edition), and to Johnny O'Toole who managed the relationship with Amazon and answered many of my questions. Thanks to Marie Beaugureau, Ben Lorica, Mike Loukides, and Laurel Ruma for believing in this project and helping me define its scope. Thanks to Matt Hacker and all of the Atlas team for answering all my technical questions regarding formatting, AsciiDoc, and LaTeX, and thanks to Nick Adams, Rebecca Demarest, Rachel Head, Judith McConville, Helen Monroe, Karen Montgomery, Rachel Roumeliotis, and everyone else at O'Reilly who contributed to this book.

I would also like to thank my former Google colleagues, in particular the YouTube video classification team, for teaching me so much about Machine Learning. I could never have started the first edition without them. Special thanks to my personal ML gurus: Clément Courbet, Julien Dubois, Mathias Kende, Daniel Kitachewsky, James Pack, Alexander Pak, Anosh Raj, Vitor Sessak, Wiktor Tomczak, Ingrid von Glehn, and Rich Washington. And thanks to everyone else I worked with at YouTube and in the amazing Google research teams in Mountain View. Many thanks as well to Martin Andrews, Sam Witteveen, and Jason Zaman for welcoming me into their Google Developer Experts group in Singapore, with the kind support of Soonson Kwon, and for all the great discussions we had about Deep Learning and TensorFlow. Anyone interested in Deep Learning in Singapore should definitely join their [Deep Learning Singapore meetup](#). Jason deserves special thanks for sharing some of his TFLite expertise for Chapter 19!

I will never forget the kind people who reviewed the first edition of this book, including David Andrzejewski, Lukas Biewald, Justin Francis, Vincent Guilbeau, Eddy Hung, Karim Matrah, Grégoire Mesnil, Salim Sémaoune, Iain Smears, Michel Tessier, Ingrid von Glehn, Pete Warden, and of course my dear brother Sylvain. Special thanks to Haesun Park, who gave me plenty of excellent feedback and caught several errors while he was writing the Korean translation of the first edition of this book. He also translated the Jupyter notebooks into Korean, not to mention TensorFlow's documentation. I do not speak Korean, but judging by the quality of his feedback, all his translations must be truly excellent! Haesun also kindly contributed some of the solutions to the exercises in this second edition.

Last but not least, I am infinitely grateful to my beloved wife, Emmanuelle, and to our three wonderful children, Alexandre, Rémi, and Gabrielle, for encouraging me to work hard on this book. I'm also thankful to them for their insatiable curiosity: explaining some of the most difficult concepts in this book to my wife and children helped me clarify my thoughts and directly improved many parts of it. And they keep bringing me cookies and coffee! What more can one dream of?

PART I

The Fundamentals of Machine Learning

The Machine Learning Landscape

When most people hear “Machine Learning,” they picture a robot: a dependable butler or a deadly Terminator, depending on who you ask. But Machine Learning is not just a futuristic fantasy; it’s already here. In fact, it has been around for decades in some specialized applications, such as Optical Character Recognition (OCR). But the first ML application that really became mainstream, improving the lives of hundreds of millions of people, took over the world back in the 1990s: the *spam filter*. It’s not exactly a self-aware Skynet, but it does technically qualify as Machine Learning (it has actually learned so well that you seldom need to flag an email as spam anymore). It was followed by hundreds of ML applications that now quietly power hundreds of products and features that you use regularly, from better recommendations to voice search.

Where does Machine Learning start and where does it end? What exactly does it mean for a machine to *learn* something? If I download a copy of Wikipedia, has my computer really learned something? Is it suddenly smarter? In this chapter we will start by clarifying what Machine Learning is and why you may want to use it.

Then, before we set out to explore the Machine Learning continent, we will take a look at the map and learn about the main regions and the most notable landmarks: supervised versus unsupervised learning, online versus batch learning, instance-based versus model-based learning. Then we will look at the workflow of a typical ML project, discuss the main challenges you may face, and cover how to evaluate and fine-tune a Machine Learning system.

This chapter introduces a lot of fundamental concepts (and jargon) that every data scientist should know by heart. It will be a high-level overview (it’s the only chapter without much code), all rather simple, but you should make sure everything is crystal clear to you before continuing on to the rest of the book. So grab a coffee and let’s get started!



If you already know all the Machine Learning basics, you may want to skip directly to **Chapter 2**. If you are not sure, try to answer all the questions listed at the end of the chapter before moving on.

What Is Machine Learning?

Machine Learning is the science (and art) of programming computers so they can *learn from data*.

Here is a slightly more general definition:

[Machine Learning is the] field of study that gives computers the ability to learn without being explicitly programmed.

—Arthur Samuel, 1959

And a more engineering-oriented one:

A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E .

—Tom Mitchell, 1997

Your spam filter is a Machine Learning program that, given examples of spam emails (e.g., flagged by users) and examples of regular (nospam, also called “ham”) emails, can learn to flag spam. The examples that the system uses to learn are called the *training set*. Each training example is called a *training instance* (or *sample*). In this case, the task T is to flag spam for new emails, the experience E is the *training data*, and the performance measure P needs to be defined; for example, you can use the ratio of correctly classified emails. This particular performance measure is called *accuracy*, and it is often used in classification tasks.

If you just download a copy of Wikipedia, your computer has a lot more data, but it is not suddenly better at any task. Thus, downloading a copy of Wikipedia is not Machine Learning.

Why Use Machine Learning?

Consider how you would write a spam filter using traditional programming techniques (**Figure 1-1**):

1. First you would consider what spam typically looks like. You might notice that some words or phrases (such as “4U,” “credit card,” “free,” and “amazing”) tend to come up a lot in the subject line. Perhaps you would also notice a few other patterns in the sender’s name, the email’s body, and other parts of the email.

2. You would write a detection algorithm for each of the patterns that you noticed, and your program would flag emails as spam if a number of these patterns were detected.
3. You would test your program and repeat steps 1 and 2 until it was good enough to launch.

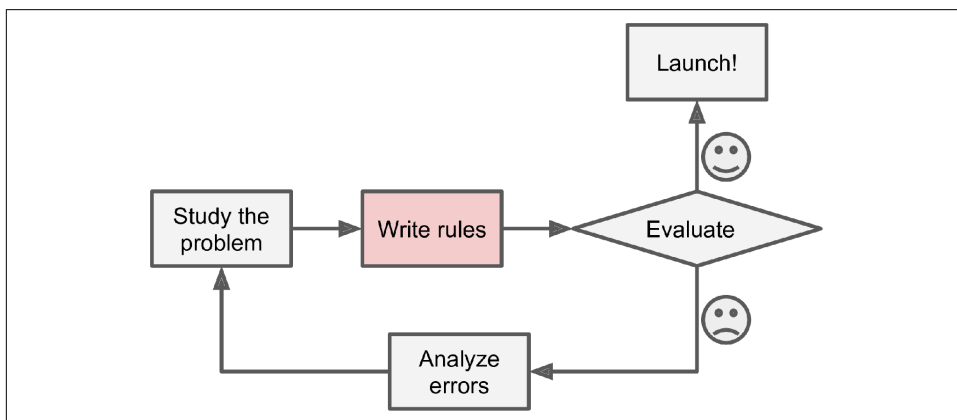


Figure 1-1. The traditional approach

Since the problem is difficult, your program will likely become a long list of complex rules—pretty hard to maintain.

In contrast, a spam filter based on Machine Learning techniques automatically learns which words and phrases are good predictors of spam by detecting unusually frequent patterns of words in the spam examples compared to the ham examples (Figure 1-2). The program is much shorter, easier to maintain, and most likely more accurate.

What if spammers notice that all their emails containing “4U” are blocked? They might start writing “For U” instead. A spam filter using traditional programming techniques would need to be updated to flag “For U” emails. If spammers keep working around your spam filter, you will need to keep writing new rules forever.

In contrast, a spam filter based on Machine Learning techniques automatically notices that “For U” has become unusually frequent in spam flagged by users, and it starts flagging them without your intervention (Figure 1-3).

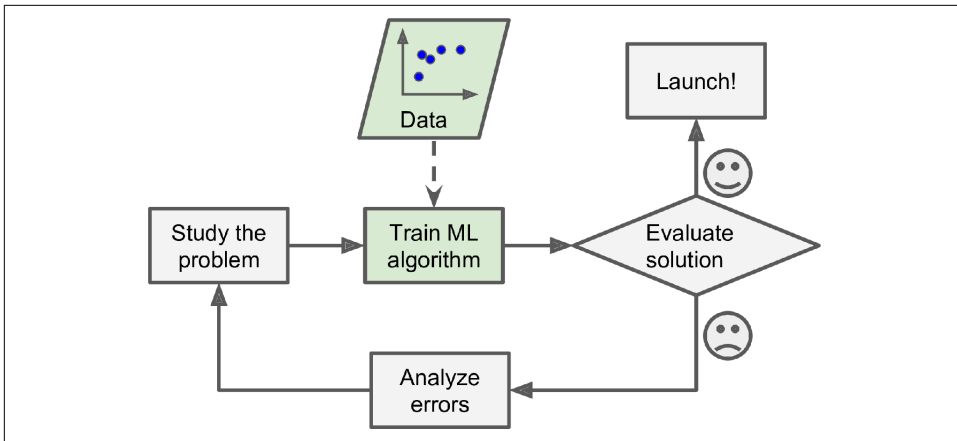


Figure 1-2. The Machine Learning approach

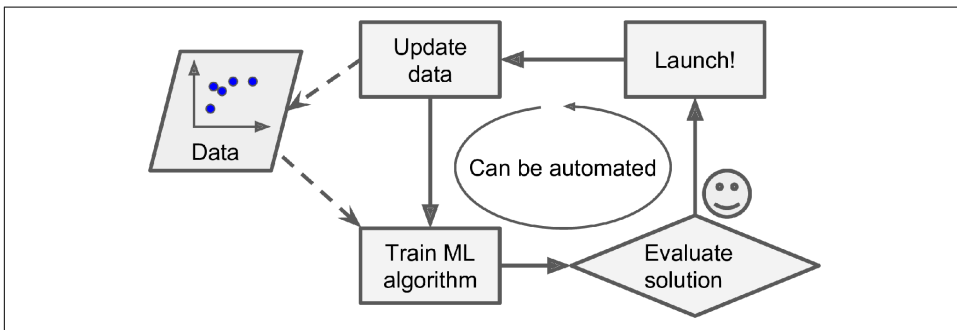


Figure 1-3. Automatically adapting to change

Another area where Machine Learning shines is for problems that either are too complex for traditional approaches or have no known algorithm. For example, consider speech recognition. Say you want to start simple and write a program capable of distinguishing the words “one” and “two.” You might notice that the word “two” starts with a high-pitch sound (“T”), so you could hardcode an algorithm that measures high-pitch sound intensity and use that to distinguish ones and twos—but obviously this technique will not scale to thousands of words spoken by millions of very different people in noisy environments and in dozens of languages. The best solution (at least today) is to write an algorithm that learns by itself, given many example recordings for each word.

Finally, Machine Learning can help humans learn (Figure 1-4). ML algorithms can be inspected to see what they have learned (although for some algorithms this can be tricky). For instance, once a spam filter has been trained on enough spam, it can easily be inspected to reveal the list of words and combinations of words that it believes are the best predictors of spam. Sometimes this will reveal unsuspected

correlations or new trends, and thereby lead to a better understanding of the problem. Applying ML techniques to dig into large amounts of data can help discover patterns that were not immediately apparent. This is called *data mining*.

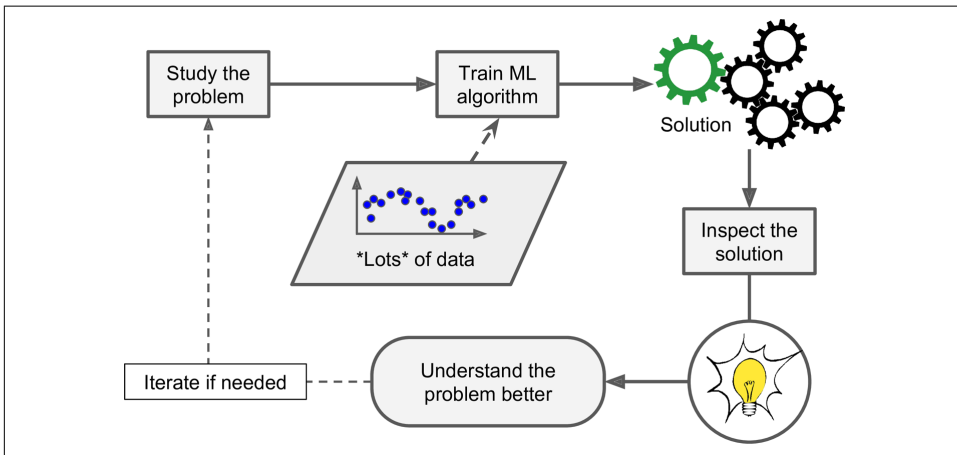


Figure 1-4. Machine Learning can help humans learn

To summarize, Machine Learning is great for:

- Problems for which existing solutions require a lot of fine-tuning or long lists of rules: one Machine Learning algorithm can often simplify code and perform better than the traditional approach.
- Complex problems for which using a traditional approach yields no good solution: the best Machine Learning techniques can perhaps find a solution.
- Fluctuating environments: a Machine Learning system can adapt to new data.
- Getting insights about complex problems and large amounts of data.

Examples of Applications

Let's look at some concrete examples of Machine Learning tasks, along with the techniques that can tackle them:

Analyzing images of products on a production line to automatically classify them

This is image classification, typically performed using convolutional neural networks (CNNs; see [Chapter 14](#)).

Detecting tumors in brain scans

This is semantic segmentation, where each pixel in the image is classified (as we want to determine the exact location and shape of tumors), typically using CNNs as well.

Automatically classifying news articles

This is natural language processing (NLP), and more specifically text classification, which can be tackled using recurrent neural networks (RNNs), CNNs, or Transformers (see [Chapter 16](#)).

Automatically flagging offensive comments on discussion forums

This is also text classification, using the same NLP tools.

Summarizing long documents automatically

This is a branch of NLP called text summarization, again using the same tools.

Creating a chatbot or a personal assistant

This involves many NLP components, including natural language understanding (NLU) and question-answering modules.

Forecasting your company's revenue next year, based on many performance metrics

This is a regression task (i.e., predicting values) that may be tackled using any regression model, such as a Linear Regression or Polynomial Regression model (see [Chapter 4](#)), a regression SVM (see [Chapter 5](#)), a regression Random Forest (see [Chapter 7](#)), or an artificial neural network (see [Chapter 10](#)). If you want to take into account sequences of past performance metrics, you may want to use RNNs, CNNs, or Transformers (see [Chapters 15 and 16](#)).

Making your app react to voice commands

This is speech recognition, which requires processing audio samples: since they are long and complex sequences, they are typically processed using RNNs, CNNs, or Transformers (see [Chapters 15 and 16](#)).

Detecting credit card fraud

This is anomaly detection (see [Chapter 9](#)).

Segmenting clients based on their purchases so that you can design a different marketing strategy for each segment

This is clustering (see [Chapter 9](#)).

Representing a complex, high-dimensional dataset in a clear and insightful diagram

This is data visualization, often involving dimensionality reduction techniques (see [Chapter 8](#)).

Recommending a product that a client may be interested in, based on past purchases

This is a recommender system. One approach is to feed past purchases (and other information about the client) to an artificial neural network (see [Chap-](#)

ter 10), and get it to output the most likely next purchase. This neural net would typically be trained on past sequences of purchases across all clients.

Building an intelligent bot for a game

This is often tackled using Reinforcement Learning (RL; see [Chapter 18](#)), which is a branch of Machine Learning that trains agents (such as bots) to pick the actions that will maximize their rewards over time (e.g., a bot may get a reward every time the player loses some life points), within a given environment (such as the game). The famous AlphaGo program that beat the world champion at the game of Go was built using RL.

This list could go on and on, but hopefully it gives you a sense of the incredible breadth and complexity of the tasks that Machine Learning can tackle, and the types of techniques that you would use for each task.

Types of Machine Learning Systems

There are so many different types of Machine Learning systems that it is useful to classify them in broad categories, based on the following criteria:

- Whether or not they are trained with human supervision (supervised, unsupervised, semisupervised, and Reinforcement Learning)
- Whether or not they can learn incrementally on the fly (online versus batch learning)
- Whether they work by simply comparing new data points to known data points, or instead by detecting patterns in the training data and building a predictive model, much like scientists do (instance-based versus model-based learning)

These criteria are not exclusive; you can combine them in any way you like. For example, a state-of-the-art spam filter may learn on the fly using a deep neural network model trained using examples of spam and ham; this makes it an online, model-based, supervised learning system.

Let's look at each of these criteria a bit more closely.

Supervised/Unsupervised Learning

Machine Learning systems can be classified according to the amount and type of supervision they get during training. There are four major categories: supervised learning, unsupervised learning, semisupervised learning, and Reinforcement Learning.

Supervised learning

In *supervised learning*, the training set you feed to the algorithm includes the desired solutions, called *labels* (Figure 1-5).

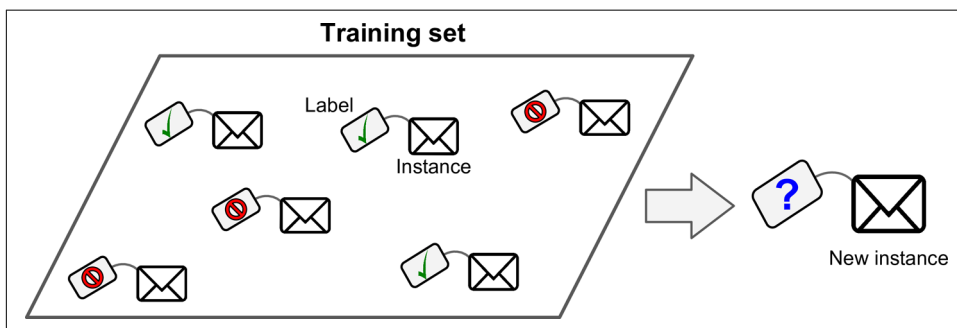


Figure 1-5. A labeled training set for spam classification (an example of supervised learning)

A typical supervised learning task is *classification*. The spam filter is a good example of this: it is trained with many example emails along with their *class* (spam or ham), and it must learn how to classify new emails.

Another typical task is to predict a *target* numeric value, such as the price of a car, given a set of *features* (mileage, age, brand, etc.) called *predictors*. This sort of task is called *regression* (Figure 1-6).¹ To train the system, you need to give it many examples of cars, including both their predictors and their labels (i.e., their prices).



In Machine Learning an *attribute* is a data type (e.g., “mileage”), while a *feature* has several meanings, depending on the context, but generally means an attribute plus its value (e.g., “mileage = 15,000”). Many people use the words *attribute* and *feature* interchangeably.

Note that some regression algorithms can be used for classification as well, and vice versa. For example, *Logistic Regression* is commonly used for classification, as it can output a value that corresponds to the probability of belonging to a given class (e.g., 20% chance of being spam).

¹ Fun fact: this odd-sounding name is a statistics term introduced by Francis Galton while he was studying the fact that the children of tall people tend to be shorter than their parents. Since the children were shorter, he called this *regression to the mean*. This name was then applied to the methods he used to analyze correlations between variables.



Figure 1-6. A regression problem: predict a value, given an input feature (there are usually multiple input features, and sometimes multiple output values)

Here are some of the most important supervised learning algorithms (covered in this book):

- k-Nearest Neighbors
- Linear Regression
- Logistic Regression
- Support Vector Machines (SVMs)
- Decision Trees and Random Forests
- Neural networks²

Unsupervised learning

In *unsupervised learning*, as you might guess, the training data is unlabeled (Figure 1-7). The system tries to learn without a teacher.

² Some neural network architectures can be unsupervised, such as autoencoders and restricted Boltzmann machines. They can also be semisupervised, such as in deep belief networks and unsupervised pretraining.

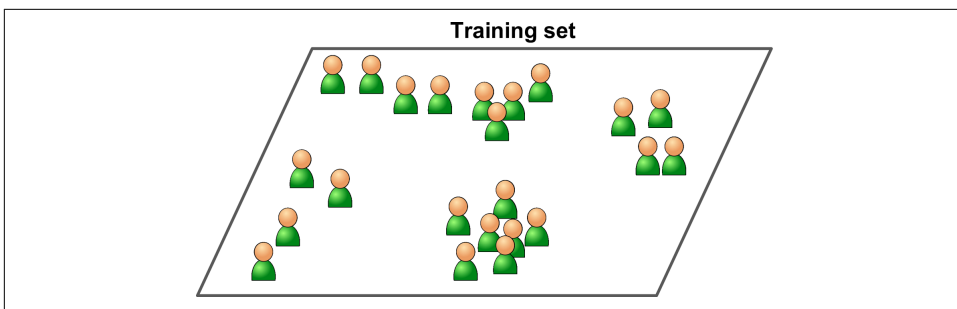


Figure 1-7. An unlabeled training set for unsupervised learning

Here are some of the most important unsupervised learning algorithms (most of these are covered in Chapters 8 and 9):

- Clustering
 - K-Means
 - DBSCAN
 - Hierarchical Cluster Analysis (HCA)
- Anomaly detection and novelty detection
 - One-class SVM
 - Isolation Forest
- Visualization and dimensionality reduction
 - Principal Component Analysis (PCA)
 - Kernel PCA
 - Locally Linear Embedding (LLE)
 - t-Distributed Stochastic Neighbor Embedding (t-SNE)
- Association rule learning
 - Apriori
 - Eclat

For example, say you have a lot of data about your blog’s visitors. You may want to run a *clustering* algorithm to try to detect groups of similar visitors (Figure 1-8). At no point do you tell the algorithm which group a visitor belongs to: it finds those connections without your help. For example, it might notice that 40% of your visitors are males who love comic books and generally read your blog in the evening, while 20% are young sci-fi lovers who visit during the weekends. If you use a *hierarchical clustering* algorithm, it may also subdivide each group into smaller groups. This may help you target your posts for each group.

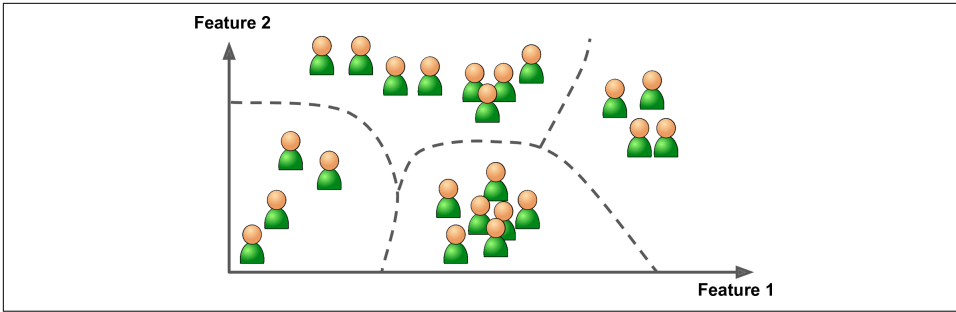


Figure 1-8. Clustering

Visualization algorithms are also good examples of unsupervised learning algorithms: you feed them a lot of complex and unlabeled data, and they output a 2D or 3D representation of your data that can easily be plotted (Figure 1-9). These algorithms try to preserve as much structure as they can (e.g., trying to keep separate clusters in the input space from overlapping in the visualization) so that you can understand how the data is organized and perhaps identify unsuspected patterns.

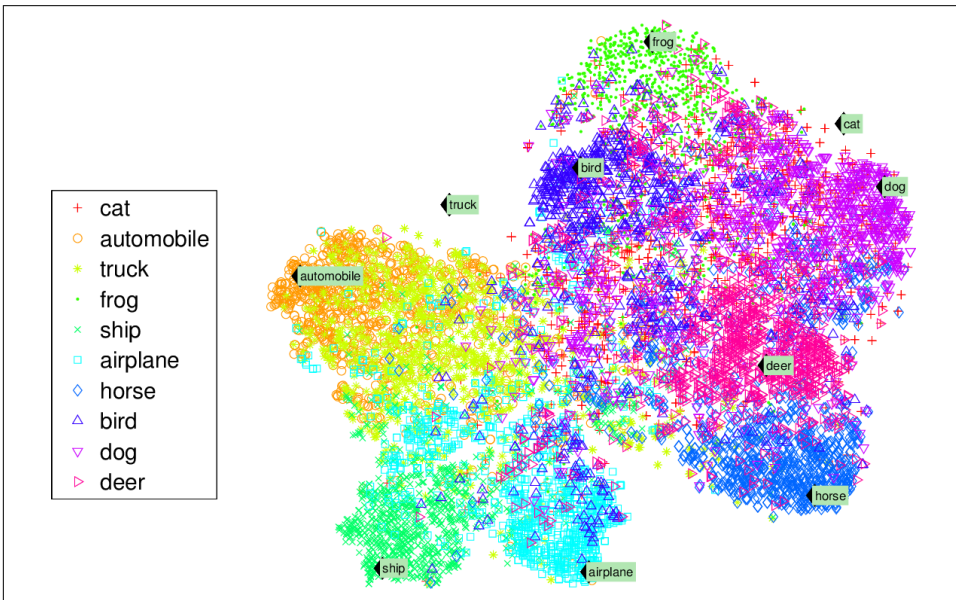


Figure 1-9. Example of a t-SNE visualization highlighting semantic clusters³

³ Notice how animals are rather well separated from vehicles and how horses are close to deer but far from birds. Figure reproduced with permission from Richard Socher et al., “Zero-Shot Learning Through Cross-Modal Transfer,” *Proceedings of the 26th International Conference on Neural Information Processing Systems 1* (2013): 935–943.

A related task is *dimensionality reduction*, in which the goal is to simplify the data without losing too much information. One way to do this is to merge several correlated features into one. For example, a car’s mileage may be strongly correlated with its age, so the dimensionality reduction algorithm will merge them into one feature that represents the car’s wear and tear. This is called *feature extraction*.



It is often a good idea to try to reduce the dimension of your training data using a dimensionality reduction algorithm before you feed it to another Machine Learning algorithm (such as a supervised learning algorithm). It will run much faster, the data will take up less disk and memory space, and in some cases it may also perform better.

Yet another important unsupervised task is *anomaly detection*—for example, detecting unusual credit card transactions to prevent fraud, catching manufacturing defects, or automatically removing outliers from a dataset before feeding it to another learning algorithm. The system is shown mostly normal instances during training, so it learns to recognize them; then, when it sees a new instance, it can tell whether it looks like a normal one or whether it is likely an anomaly (see [Figure 1-10](#)). A very similar task is *novelty detection*: it aims to detect new instances that look different from all instances in the training set. This requires having a very “clean” training set, devoid of any instance that you would like the algorithm to detect. For example, if you have thousands of pictures of dogs, and 1% of these pictures represent Chihuahuas, then a novelty detection algorithm should not treat new pictures of Chihuahuas as novelties. On the other hand, anomaly detection algorithms may consider these dogs as so rare and so different from other dogs that they would likely classify them as anomalies (no offense to Chihuahuas).

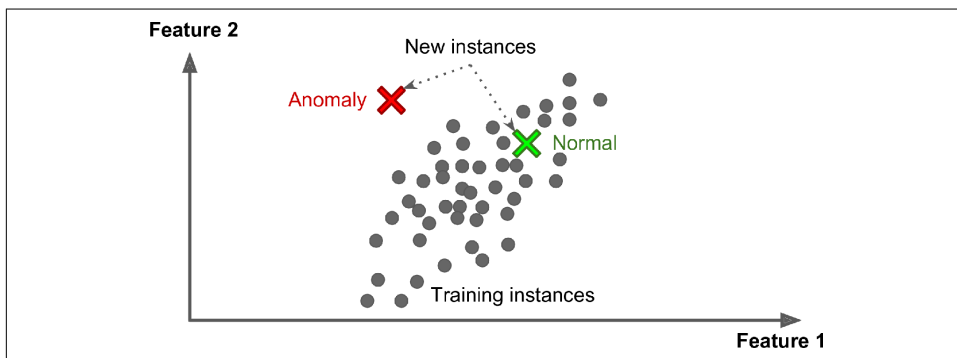


Figure 1-10. Anomaly detection

Finally, another common unsupervised task is *association rule learning*, in which the goal is to dig into large amounts of data and discover interesting relations between

attributes. For example, suppose you own a supermarket. Running an association rule on your sales logs may reveal that people who purchase barbecue sauce and potato chips also tend to buy steak. Thus, you may want to place these items close to one another.

Semisupervised learning

Since labeling data is usually time-consuming and costly, you will often have plenty of unlabeled instances, and few labeled instances. Some algorithms can deal with data that's partially labeled. This is called *semisupervised learning* (Figure 1-11).

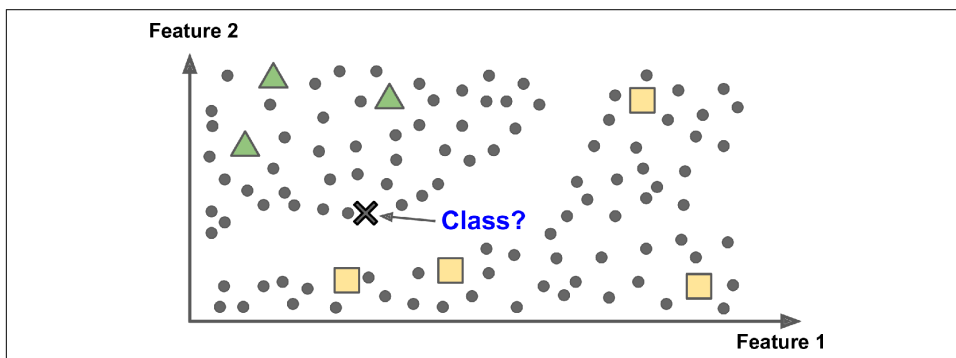


Figure 1-11. Semisupervised learning with two classes (triangles and squares): the unlabeled examples (circles) help classify a new instance (the cross) into the triangle class rather than the square class, even though it is closer to the labeled squares

Some photo-hosting services, such as Google Photos, are good examples of this. Once you upload all your family photos to the service, it automatically recognizes that the same person A shows up in photos 1, 5, and 11, while another person B shows up in photos 2, 5, and 7. This is the unsupervised part of the algorithm (clustering). Now all the system needs is for you to tell it who these people are. Just add one label per person⁴ and it is able to name everyone in every photo, which is useful for searching photos.

Most semisupervised learning algorithms are combinations of unsupervised and supervised algorithms. For example, *deep belief networks* (DBNs) are based on unsupervised components called *restricted Boltzmann machines* (RBMs) stacked on top of one another. RBMs are trained sequentially in an unsupervised manner, and then the whole system is fine-tuned using supervised learning techniques.

⁴ That's when the system works perfectly. In practice it often creates a few clusters per person, and sometimes mixes up two people who look alike, so you may need to provide a few labels per person and manually clean up some clusters.

Reinforcement Learning

Reinforcement Learning is a very different beast. The learning system, called an *agent* in this context, can observe the environment, select and perform actions, and get *rewards* in return (or *penalties* in the form of negative rewards, as shown in **Figure 1-12**). It must then learn by itself what is the best strategy, called a *policy*, to get the most reward over time. A policy defines what action the agent should choose when it is in a given situation.

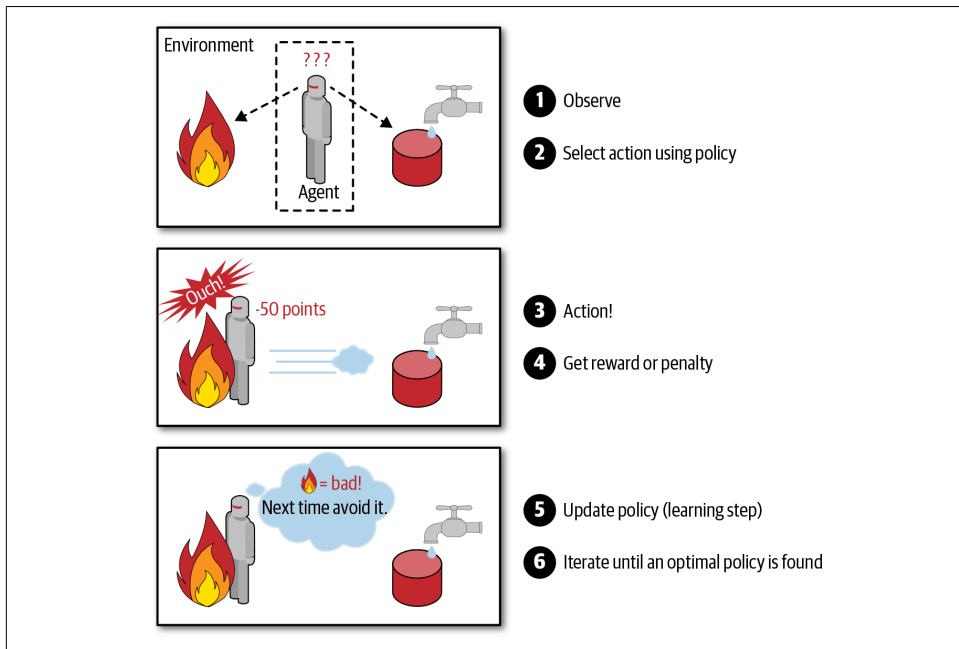


Figure 1-12. Reinforcement Learning

For example, many robots implement Reinforcement Learning algorithms to learn how to walk. DeepMind's AlphaGo program is also a good example of Reinforcement Learning: it made the headlines in May 2017 when it beat the world champion Ke Jie at the game of Go. It learned its winning policy by analyzing millions of games, and then playing many games against itself. Note that learning was turned off during the games against the champion; AlphaGo was just applying the policy it had learned.

Batch and Online Learning

Another criterion used to classify Machine Learning systems is whether or not the system can learn incrementally from a stream of incoming data.

Batch learning

In *batch learning*, the system is incapable of learning incrementally: it must be trained using all the available data. This will generally take a lot of time and computing resources, so it is typically done offline. First the system is trained, and then it is launched into production and runs without learning anymore; it just applies what it has learned. This is called *offline learning*.

If you want a batch learning system to know about new data (such as a new type of spam), you need to train a new version of the system from scratch on the full dataset (not just the new data, but also the old data), then stop the old system and replace it with the new one.

Fortunately, the whole process of training, evaluating, and launching a Machine Learning system can be automated fairly easily (as shown in [Figure 1-3](#)), so even a batch learning system can adapt to change. Simply update the data and train a new version of the system from scratch as often as needed.

This solution is simple and often works fine, but training using the full set of data can take many hours, so you would typically train a new system only every 24 hours or even just weekly. If your system needs to adapt to rapidly changing data (e.g., to predict stock prices), then you need a more reactive solution.

Also, training on the full set of data requires a lot of computing resources (CPU, memory space, disk space, disk I/O, network I/O, etc.). If you have a lot of data and you automate your system to train from scratch every day, it will end up costing you a lot of money. If the amount of data is huge, it may even be impossible to use a batch learning algorithm.

Finally, if your system needs to be able to learn autonomously and it has limited resources (e.g., a smartphone application or a rover on Mars), then carrying around large amounts of training data and taking up a lot of resources to train for hours every day is a showstopper.

Fortunately, a better option in all these cases is to use algorithms that are capable of learning incrementally.

Online learning

In *online learning*, you train the system incrementally by feeding it data instances sequentially, either individually or in small groups called *mini-batches*. Each learning step is fast and cheap, so the system can learn about new data on the fly, as it arrives (see [Figure 1-13](#)).

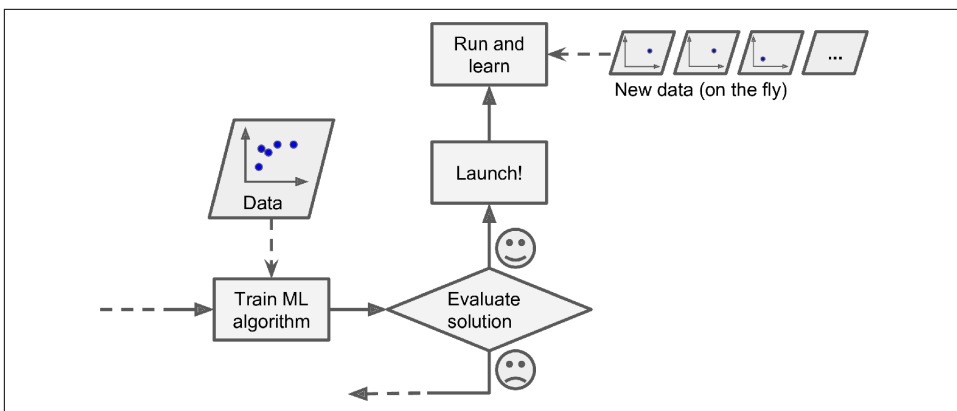


Figure 1-13. In online learning, a model is trained and launched into production, and then it keeps learning as new data comes in

Online learning is great for systems that receive data as a continuous flow (e.g., stock prices) and need to adapt to change rapidly or autonomously. It is also a good option if you have limited computing resources: once an online learning system has learned about new data instances, it does not need them anymore, so you can discard them (unless you want to be able to roll back to a previous state and “replay” the data). This can save a huge amount of space.

Online learning algorithms can also be used to train systems on huge datasets that cannot fit in one machine’s main memory (this is called *out-of-core* learning). The algorithm loads part of the data, runs a training step on that data, and repeats the process until it has run on all of the data (see Figure 1-14).



Out-of-core learning is usually done offline (i.e., not on the live system), so *online learning* can be a confusing name. Think of it as *incremental learning*.

One important parameter of online learning systems is how fast they should adapt to changing data: this is called the *learning rate*. If you set a high learning rate, then your system will rapidly adapt to new data, but it will also tend to quickly forget the old data (you don’t want a spam filter to flag only the latest kinds of spam it was shown). Conversely, if you set a low learning rate, the system will have more inertia; that is, it will learn more slowly, but it will also be less sensitive to noise in the new data or to sequences of nonrepresentative data points (outliers).

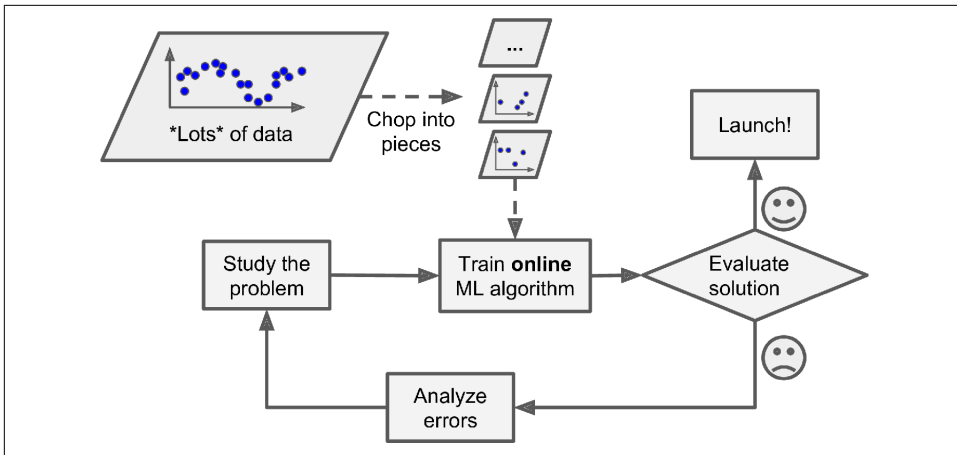


Figure 1-14. Using online learning to handle huge datasets

A big challenge with online learning is that if bad data is fed to the system, the system's performance will gradually decline. If it's a live system, your clients will notice. For example, bad data could come from a malfunctioning sensor on a robot, or from someone spamming a search engine to try to rank high in search results. To reduce this risk, you need to monitor your system closely and promptly switch learning off (and possibly revert to a previously working state) if you detect a drop in performance. You may also want to monitor the input data and react to abnormal data (e.g., using an anomaly detection algorithm).

Instance-Based Versus Model-Based Learning

One more way to categorize Machine Learning systems is by how they *generalize*. Most Machine Learning tasks are about making predictions. This means that given a number of training examples, the system needs to be able to make good predictions for (generalize to) examples it has never seen before. Having a good performance measure on the training data is good, but insufficient; the true goal is to perform well on new instances.

There are two main approaches to generalization: instance-based learning and model-based learning.

Instance-based learning

Possibly the most trivial form of learning is simply to learn by heart. If you were to create a spam filter this way, it would just flag all emails that are identical to emails that have already been flagged by users—not the worst solution, but certainly not the best.

Instead of just flagging emails that are identical to known spam emails, your spam filter could be programmed to also flag emails that are very similar to known spam emails. This requires a *measure of similarity* between two emails. A (very basic) similarity measure between two emails could be to count the number of words they have in common. The system would flag an email as spam if it has many words in common with a known spam email.

This is called *instance-based learning*: the system learns the examples by heart, then generalizes to new cases by using a similarity measure to compare them to the learned examples (or a subset of them). For example, in [Figure 1-15](#) the new instance would be classified as a triangle because the majority of the most similar instances belong to that class.

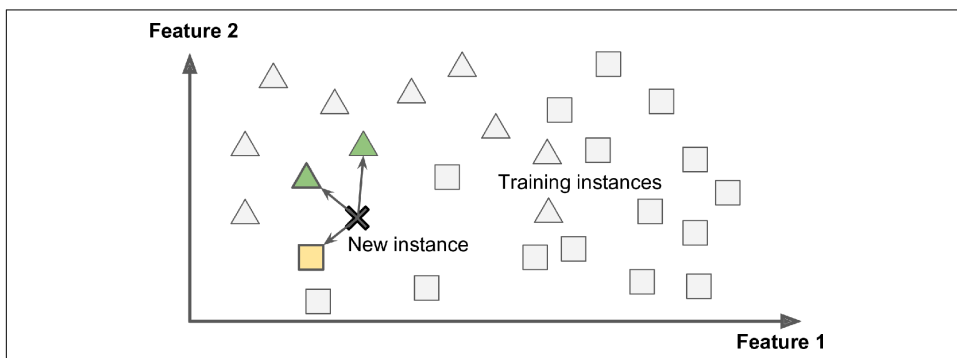


Figure 1-15. Instance-based learning

Model-based learning

Another way to generalize from a set of examples is to build a model of these examples and then use that model to make *predictions*. This is called *model-based learning* ([Figure 1-16](#)).

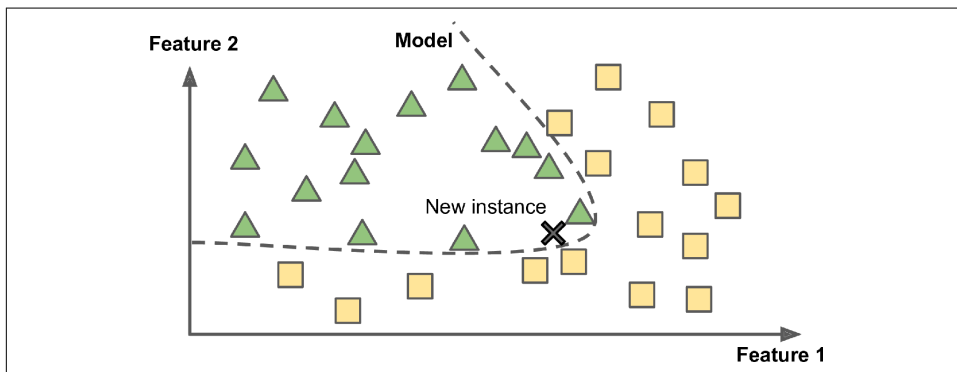


Figure 1-16. Model-based learning

For example, suppose you want to know if money makes people happy, so you download the Better Life Index data from the [OECD's website](#) and stats about gross domestic product (GDP) per capita from the [IMF's website](#). Then you join the tables and sort by GDP per capita. [Table 1-1](#) shows an excerpt of what you get.

Table 1-1. Does money make people happier?

Country	GDP per capita (USD)	Life satisfaction
Hungary	12,240	4.9
Korea	27,195	5.8
France	37,675	6.5
Australia	50,962	7.3
United States	55,805	7.2

Let's plot the data for these countries ([Figure 1-17](#)).

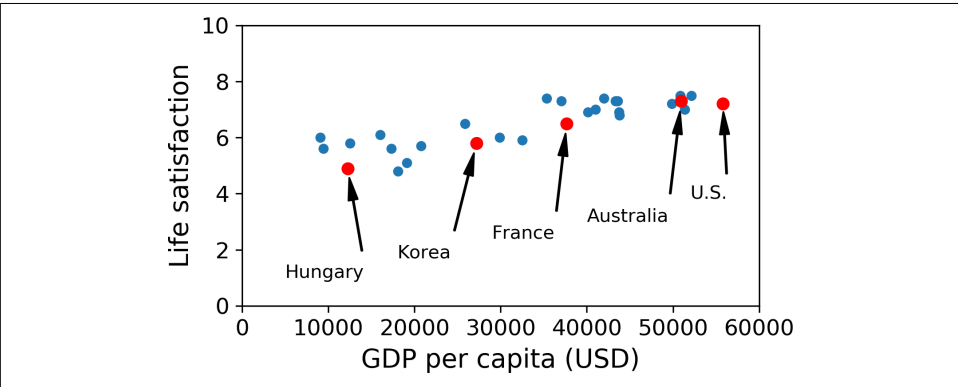


Figure 1-17. Do you see a trend here?

There does seem to be a trend here! Although the data is *noisy* (i.e., partly random), it looks like life satisfaction goes up more or less linearly as the country's GDP per capita increases. So you decide to model life satisfaction as a linear function of GDP per capita. This step is called *model selection*: you selected a *linear model* of life satisfaction with just one attribute, GDP per capita ([Equation 1-1](#)).

Equation 1-1. A simple linear model

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita}$$

This model has two *model parameters*, θ_0 and θ_1 .⁵ By tweaking these parameters, you can make your model represent any linear function, as shown in Figure 1-18.

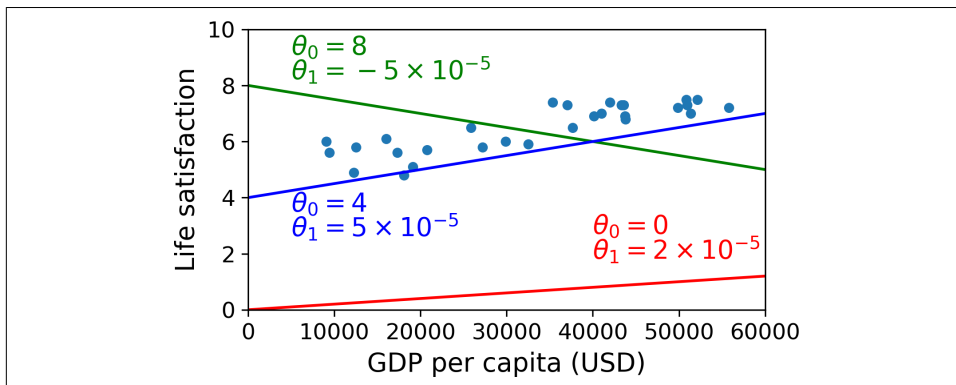


Figure 1-18. A few possible linear models

Before you can use your model, you need to define the parameter values θ_0 and θ_1 . How can you know which values will make your model perform best? To answer this question, you need to specify a performance measure. You can either define a *utility function* (or *fitness function*) that measures how *good* your model is, or you can define a *cost function* that measures how *bad* it is. For Linear Regression problems, people typically use a cost function that measures the distance between the linear model's predictions and the training examples; the objective is to minimize this distance.

This is where the Linear Regression algorithm comes in: you feed it your training examples, and it finds the parameters that make the linear model fit best to your data. This is called *training* the model. In our case, the algorithm finds that the optimal parameter values are $\theta_0 = 4.85$ and $\theta_1 = 4.91 \times 10^{-5}$.



Confusingly, the same word “model” can refer to a *type of model* (e.g., Linear Regression), to a *fully specified model architecture* (e.g., Linear Regression with one input and one output), or to the *final trained model* ready to be used for predictions (e.g., Linear Regression with one input and one output, using $\theta_0 = 4.85$ and $\theta_1 = 4.91 \times 10^{-5}$). Model selection consists in choosing the type of model and fully specifying its architecture. Training a model means running an algorithm to find the model parameters that will make it best fit the training data (and hopefully make good predictions on new data).

⁵ By convention, the Greek letter θ (theta) is frequently used to represent model parameters.